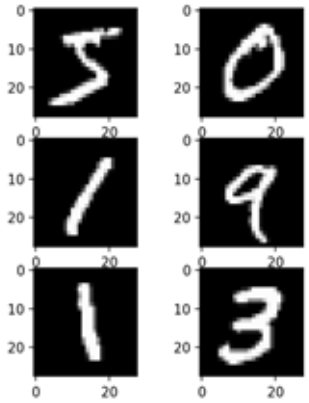
Deep Learning Course

Neural Network Essentials

Recap - We can get results without knowing anything



```
+import tensorflow as tf
+from tensorflow.keras.datasets import mnist
cell (⌘/Ctrl+Enter)                ls import Sequential
has not been executed in this session  rs import Dense, Flatten
+
+# Load the MNIST dataset
+(x_train, y_train), (x_test, y_test) = mnist.load_data()
+
+# Preprocess the data
+x_train, x_test = x_train / 255.0, x_test / 255.0
+
+# Build a simple model
+model = Sequential([
+    Flatten(input_shape=(28, 28))
+    Dense(128, activation='relu')
+    Dense(10, activation='softmax
+])
+
+# Compile the model
+model.compile(optimizer='adam',
+              loss='sparse_catego
+              metrics=['accuracy'
+
+# Train the model
+model.fit(x_train, y_train, epoch
+
+# Evaluate the model
+loss, accuracy = model.evaluate(x
+print(f"Test accuracy: {accuracy}
```

R build me a minimialist model to load the mnist digits and train a model to predict them with tensorflow. Just one train/test split is needed.

<> Empty cell

Sure, I can help with that! Here is the code to load the MNIST dataset, train a simple model, and evaluate its performance.

Accept & Run Accept Cancel

What can I help you build?

Gemini can make mistakes so double-check it and use code with caution. Learn more

Goal: Develop Understanding / Interpretation

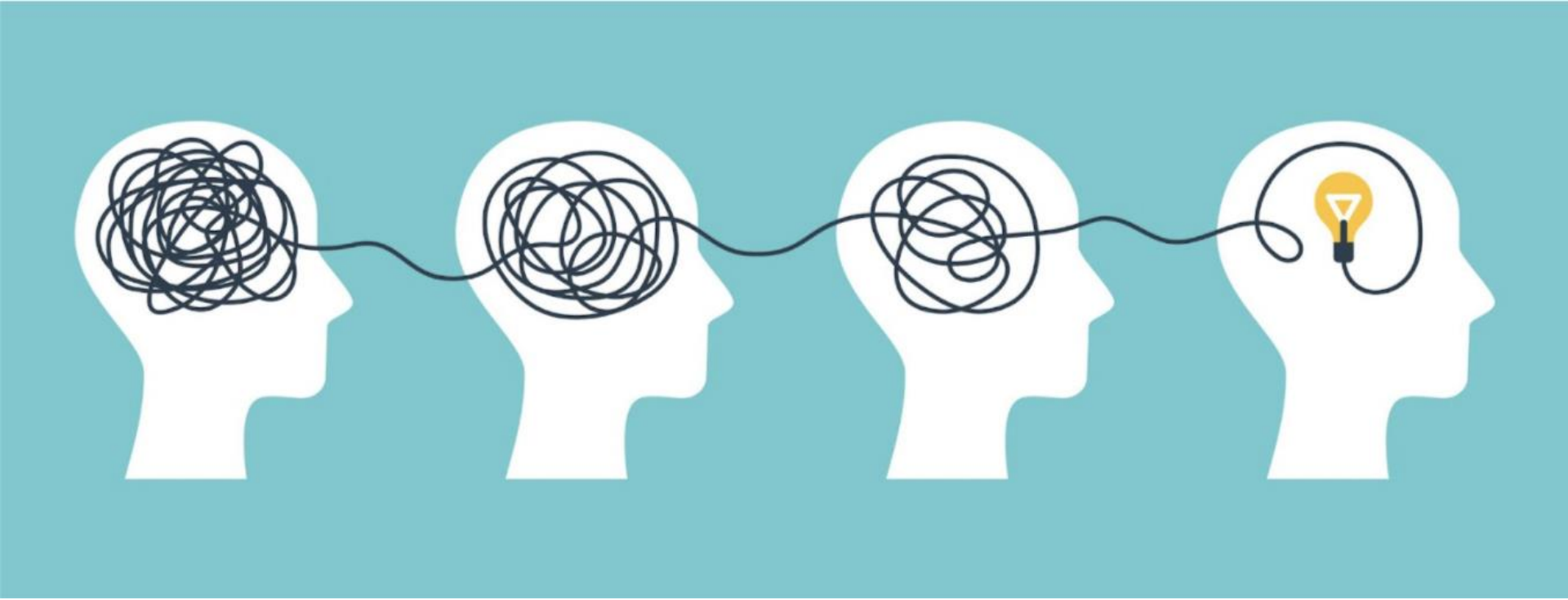
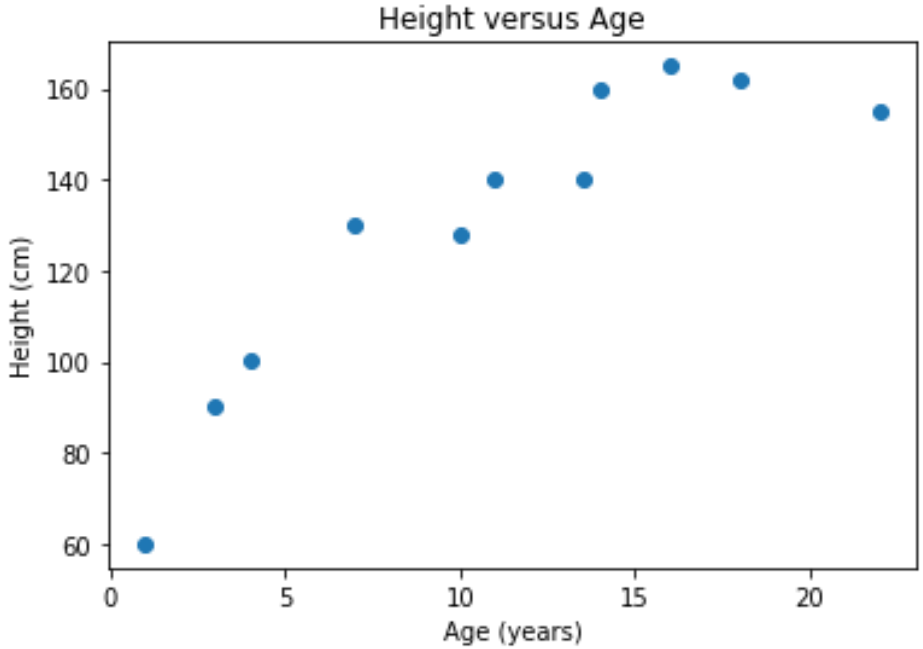


Image Credit: https://www.google.com/url?sa=i&url=https%3A%2F%2Fspotonsolutions.com%2Fmonday-mo-knowing-vs-understanding-and-why-it-matters%2F&psig=AOvVaw38ZggrgKkNhbhTNHX4y60P&ust=1757921193385000&source=images&cd=vfe&opi=89978449&ved=0CBgQjhxqFwoTCPjtiPnc148DFQAAAAAdAAAAABAE

Context – Machine Learning Basics

Regression Problem Example

X = age = [1. 3. 4. 7. 10. 11. 14. 13.5 16. 18. 22.]
Y = height = [60. 90. 100.5 130. 128. 140. 160. 140.
165. 162. 155.]



Regression and Classification

Handwaving Problem Statement: Learn to predict a Target (output) for some Features (input) based on some training data

Target = f(input)

I need to figure out the function / model f

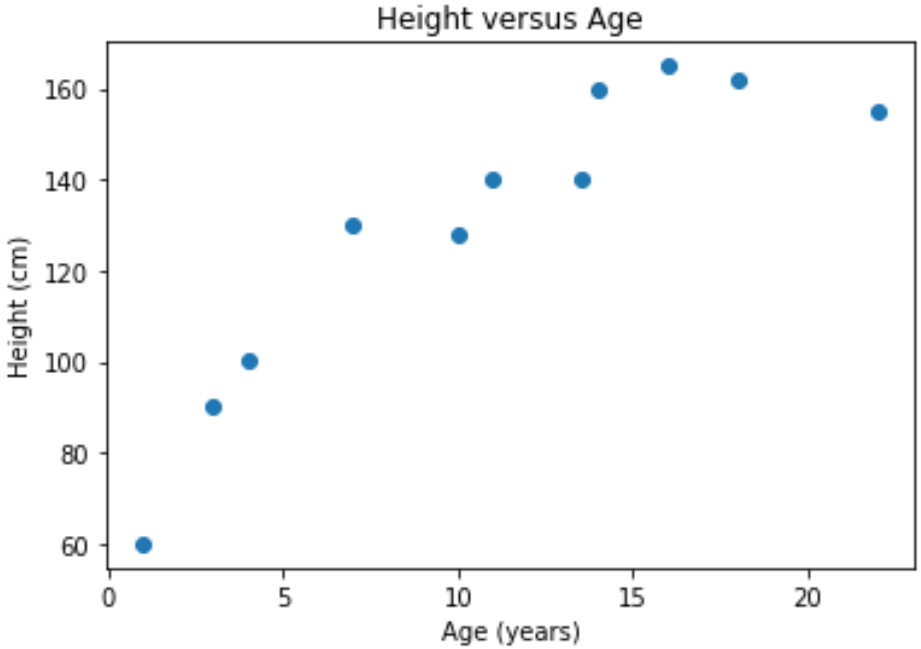
Regression: Real Valued Target (a Number)
Classification: A class target (binary, multiple types)

Target = Y = Dependent Variable
Features = X = Independent Variable

X_i = i^{th} input feature / variable

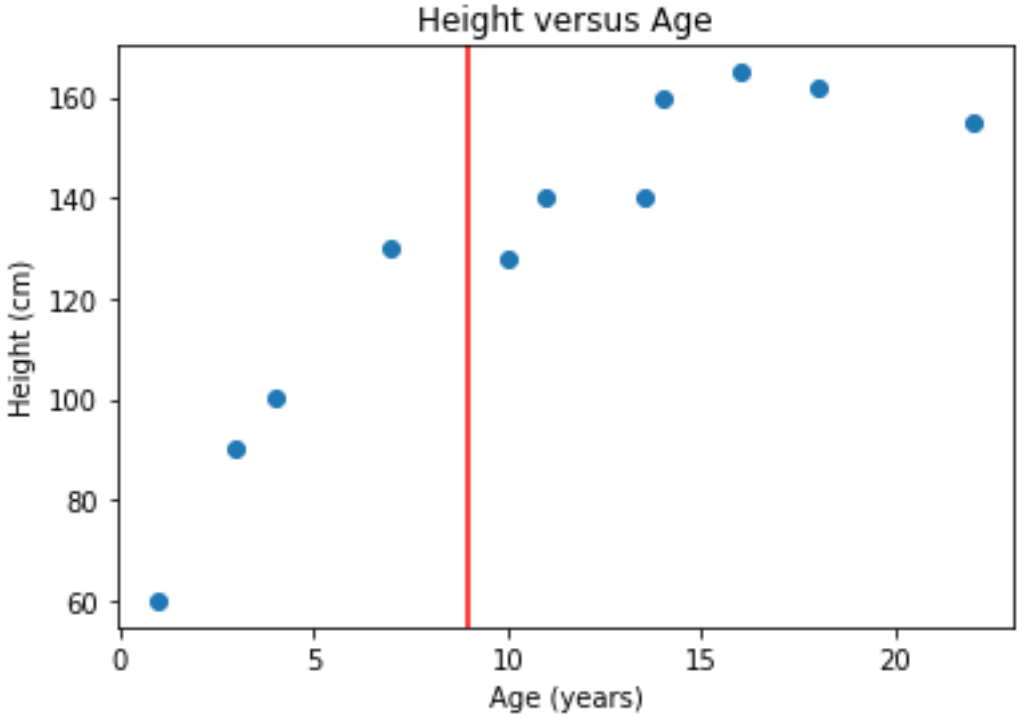
Regression Problem Example

X = age = [1. 3. 4. 7. 10. 11. 14. 13.5 16. 18. 22.]
Y = height = [60. 90. 100.5 130. 128. 140. 160. 140.
165. 162. 155.]



Intuitive Solution to the Problem

What is the likely height of a 9 year old?



Linear Functions of Data

$f(x)$

We can parameterize a linear function to model the relationship between a Target and Feature / set of features..

The most basic function we can fit to our example is a straight line of the form:

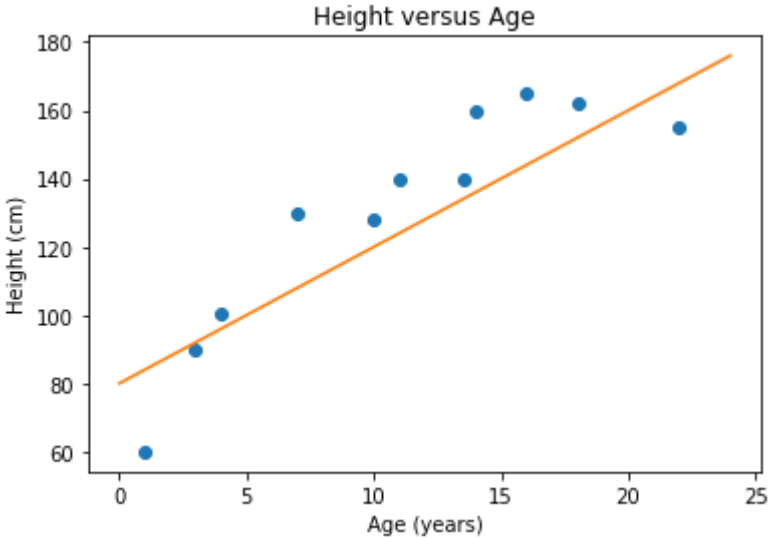
$$Y=mX+c$$

where X is our input, Y is our output, m defines the slope of the line and c defines the line's offset from the origin.

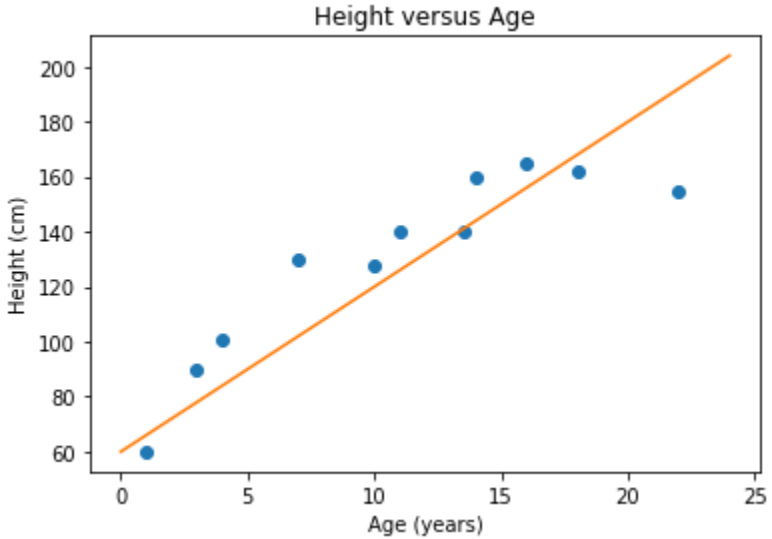
While the notation above is the one we use in school, the version we use in Machine Learning is like this:

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Example Functions against our Data

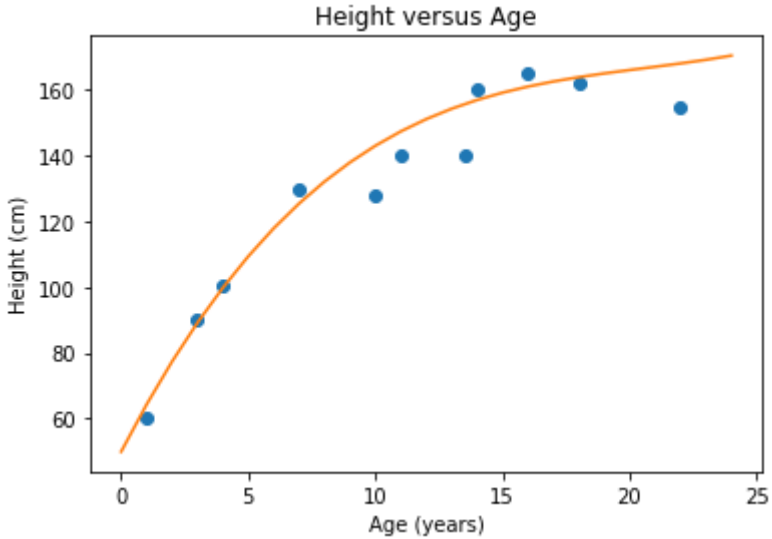


Y = 80 + 4X



Y = 60 + 6X

A Function with some higher order terms

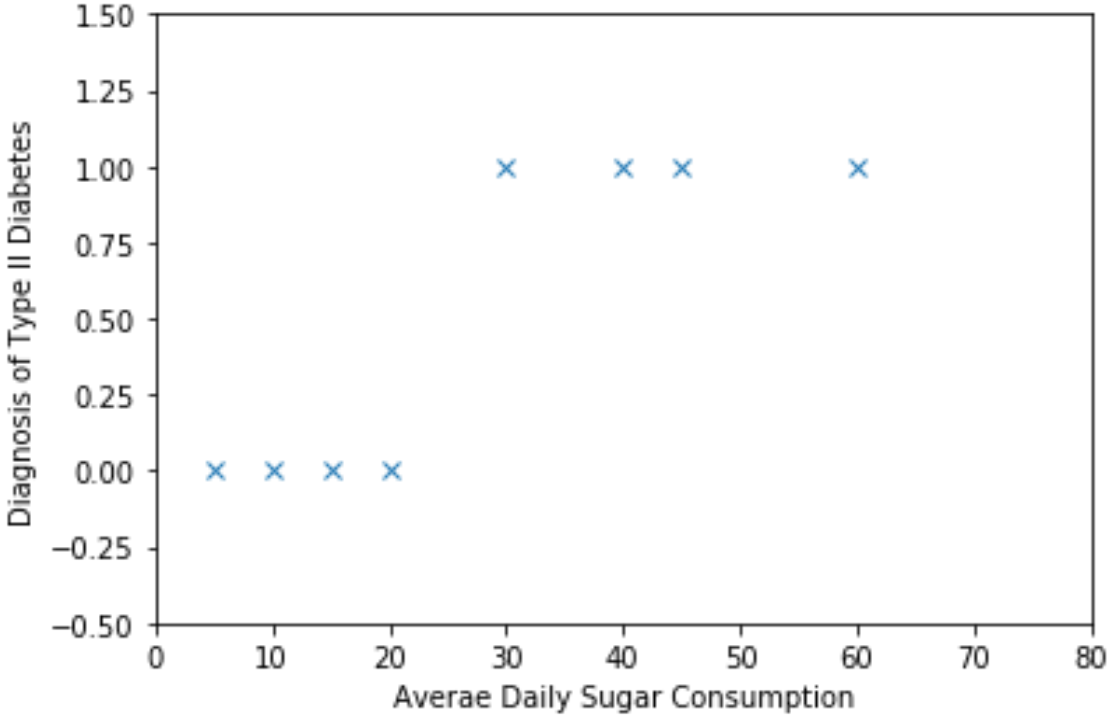


$$Y = 50 + 15X - 0.68X^2 + 0.011X^3$$

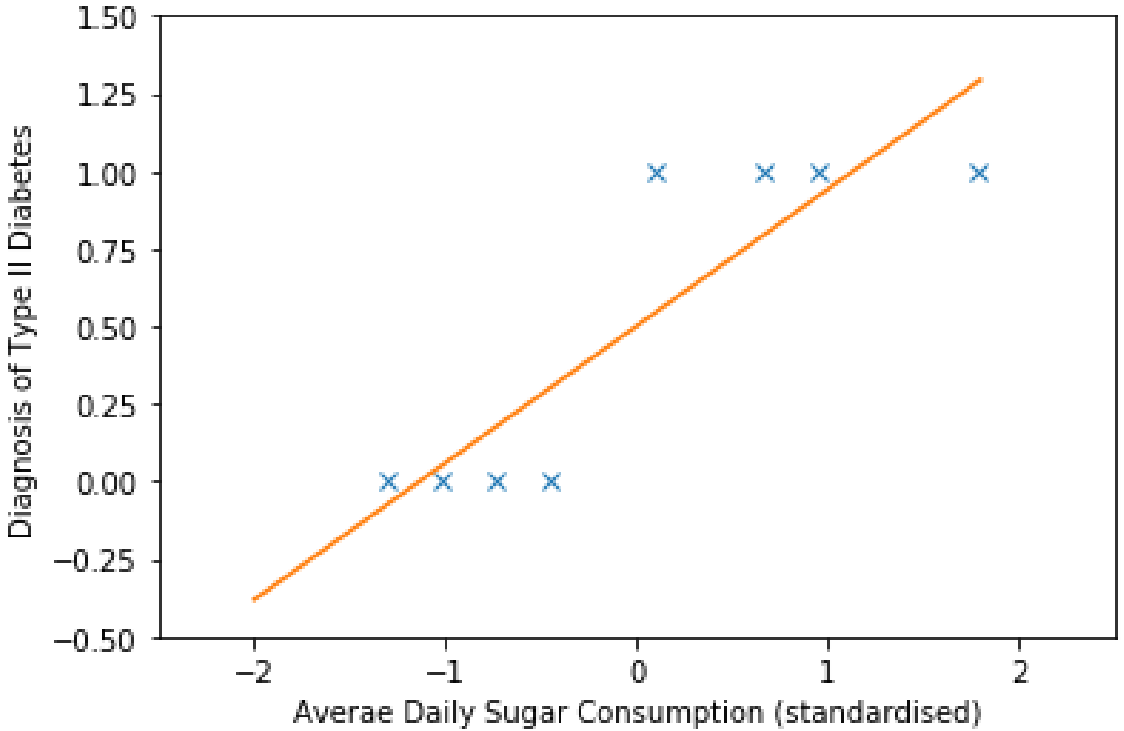
Classification Problem Statement

- Same as for Regression with the only difference being that our **Target variable is Binary** rather than real valued
  - We wish to automatically fit a function that best fits the available training data.

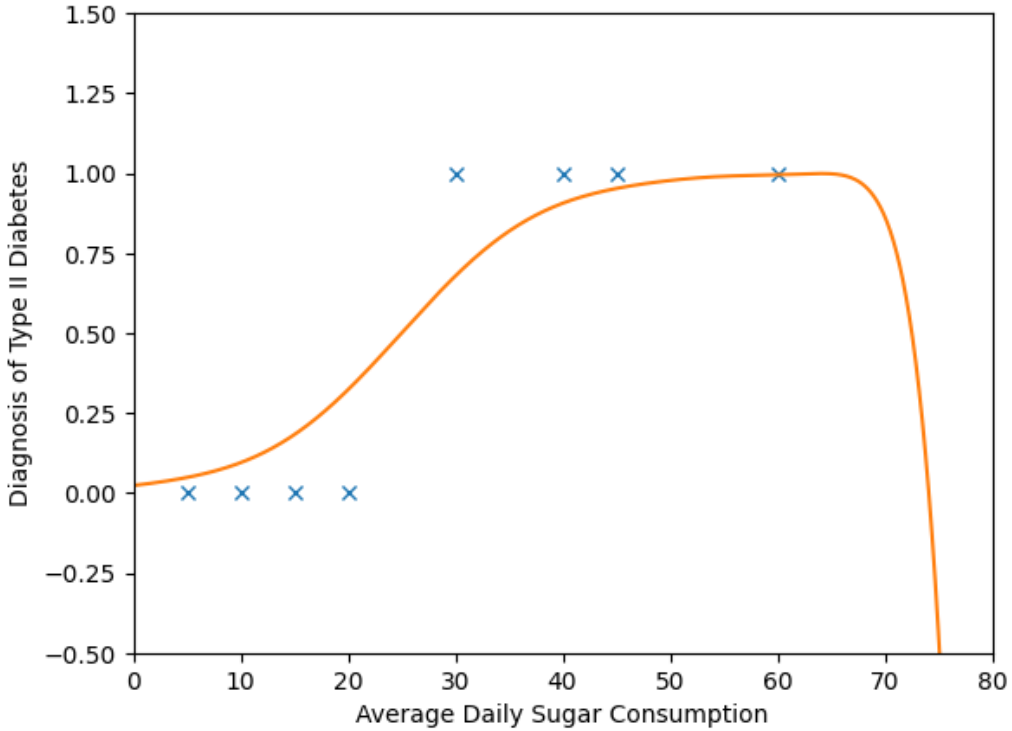
Classification - Example Problem



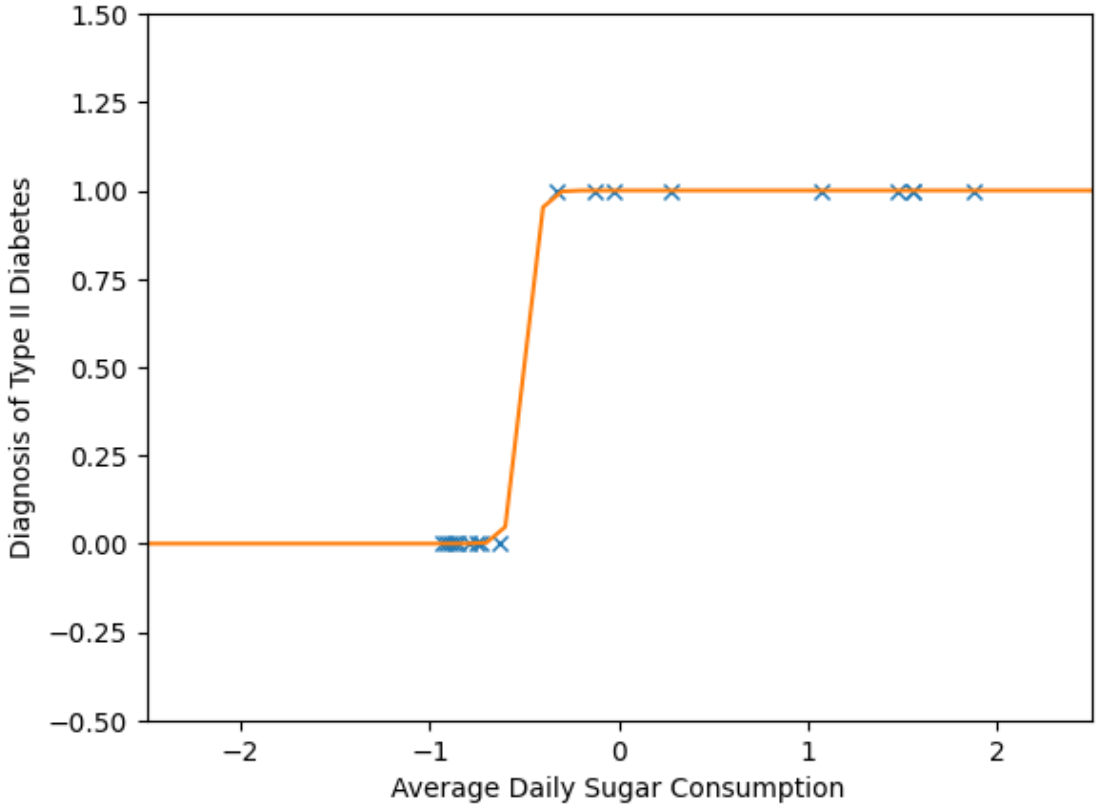
Modelling Binary Problem with Regression



Modelling Binary Problem with Regression – eek!



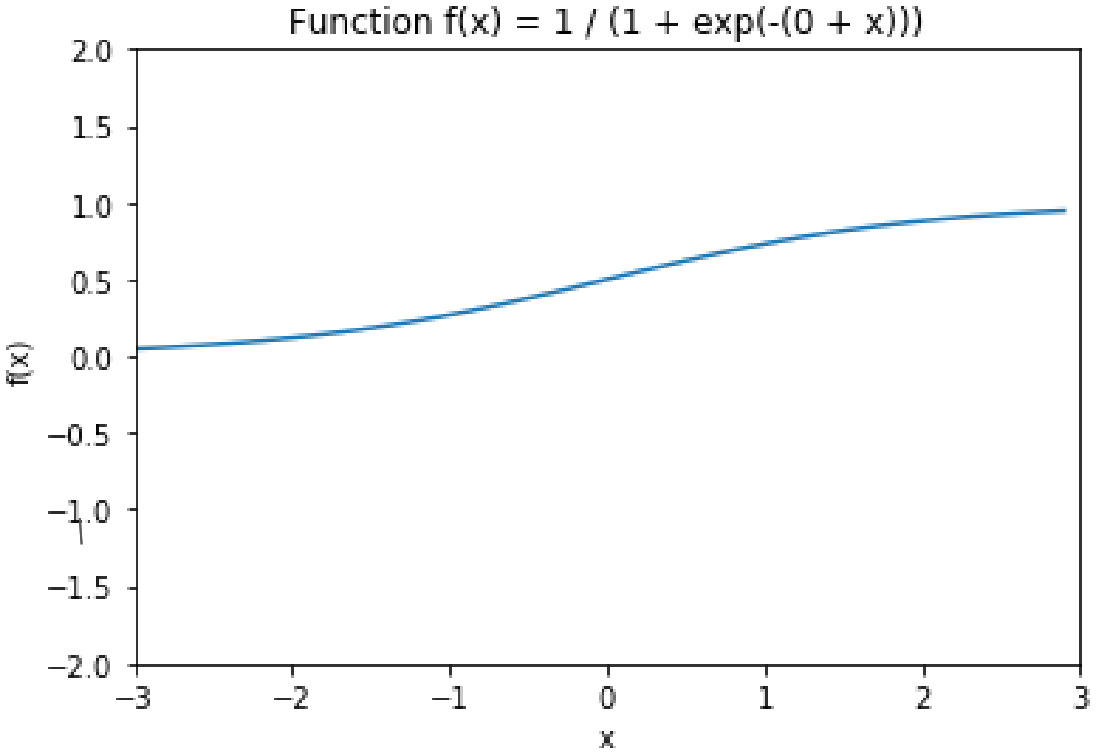
Modelling Binary Problem with Logistic Functions



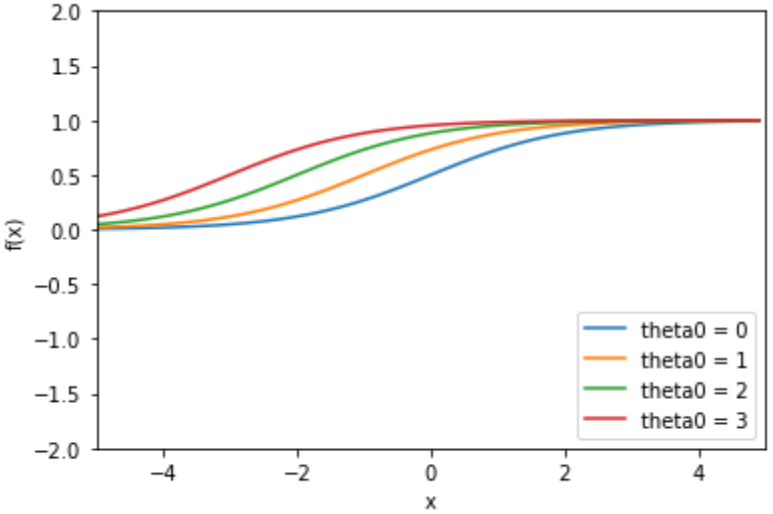
Logistic Function

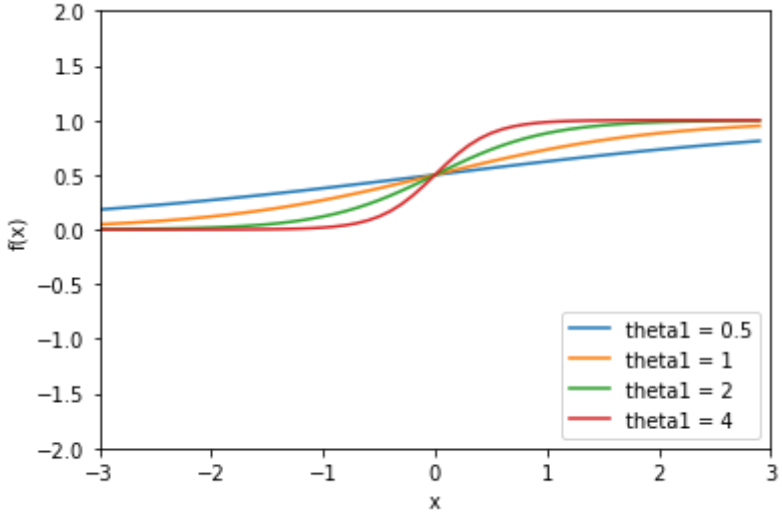
$$h_\theta(x) = \frac{1}{1 + e^{-z}}$$

$$z = \theta x = \sum_{i=0}^{n} \theta_i x_i$$

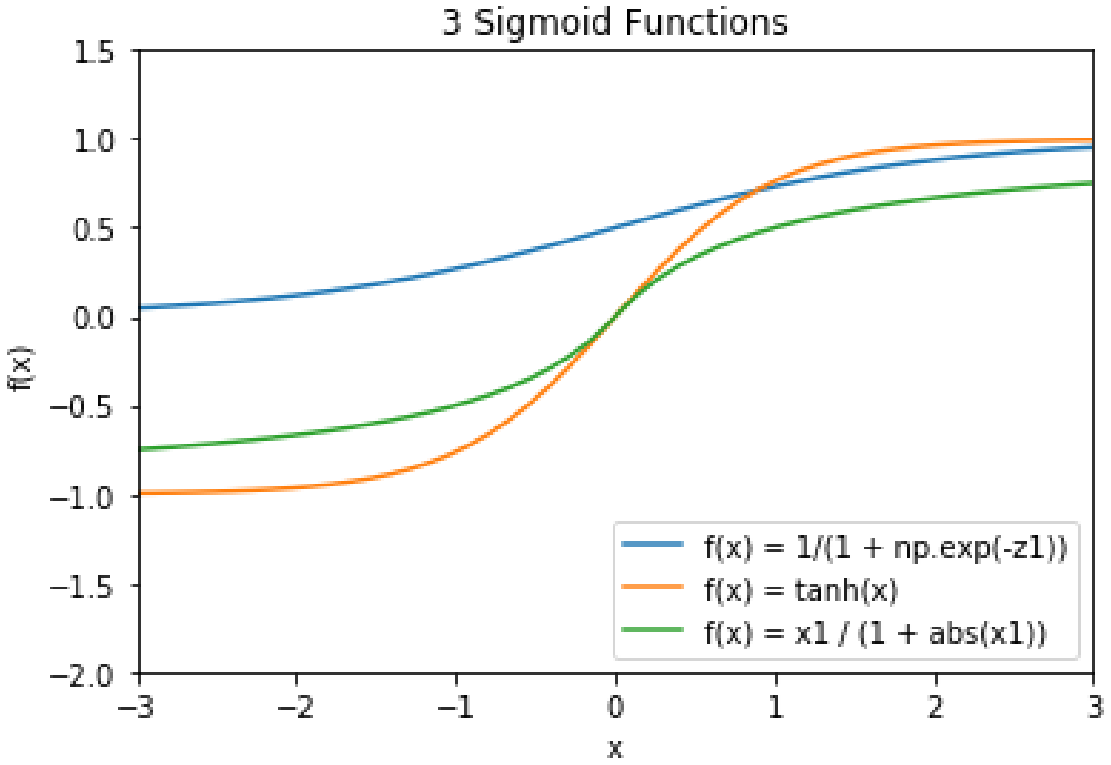


Varying the parameters

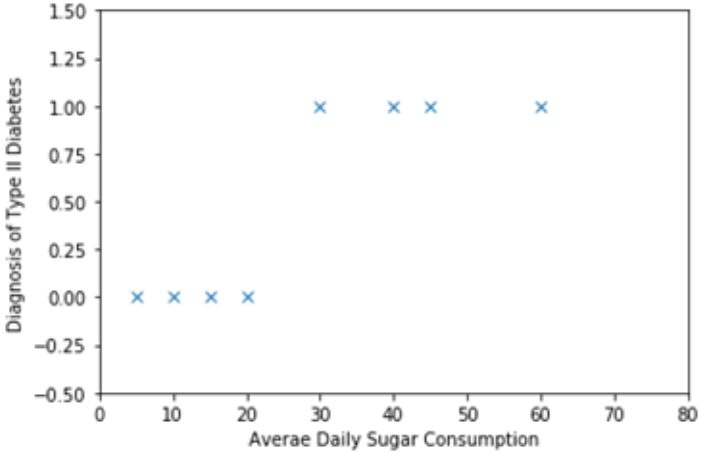




Logistic Fn as an Example of Sigmoid Functions

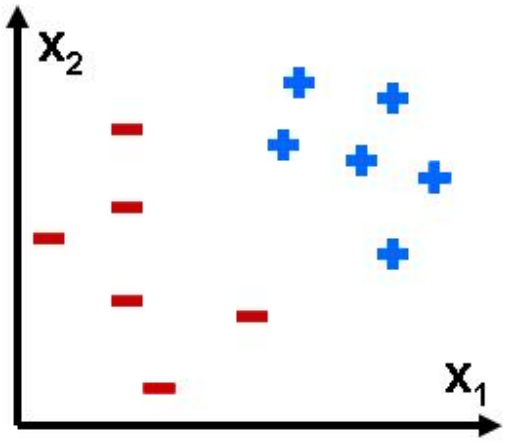


Aside: Alternative Visualisations





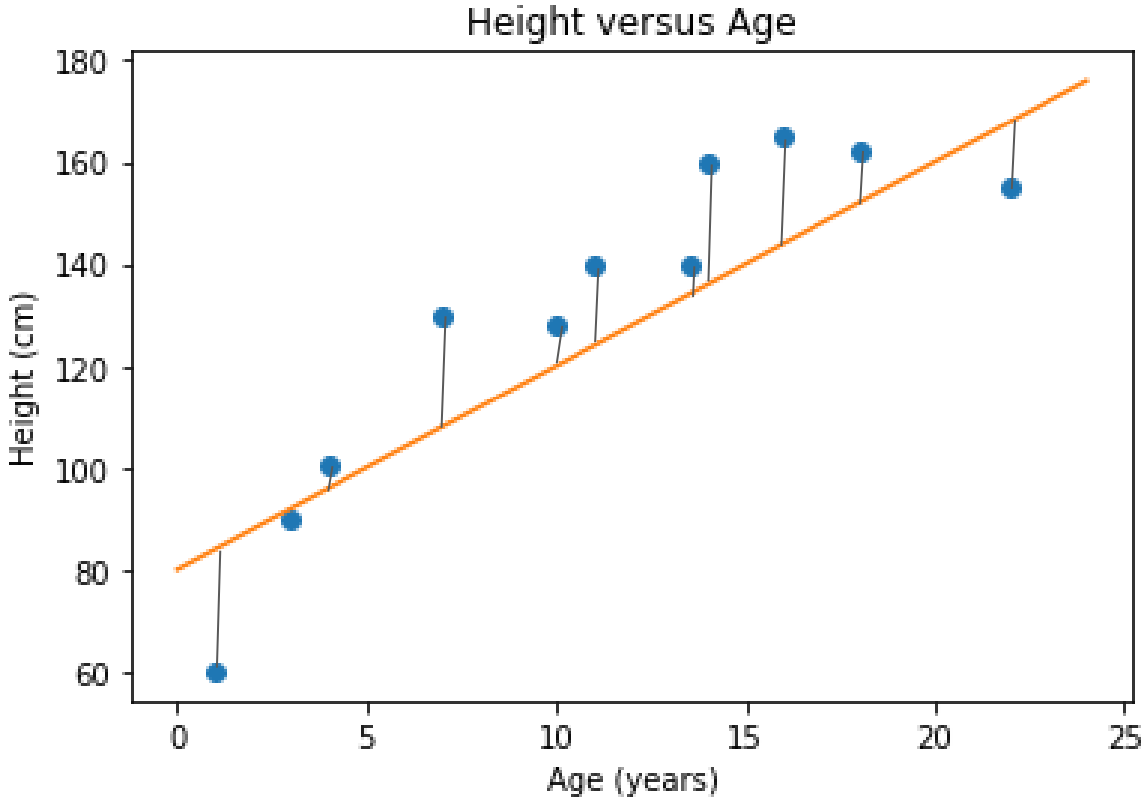
Alternate Visualizations



Measuring Error



The Error or Loss of our Functions



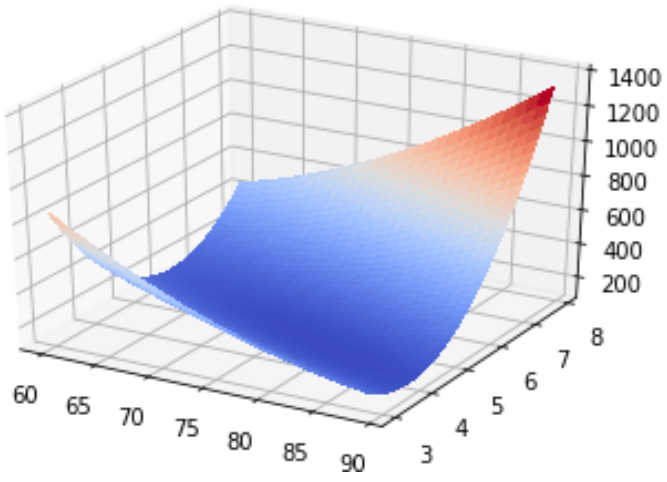
Sum of Squared Error

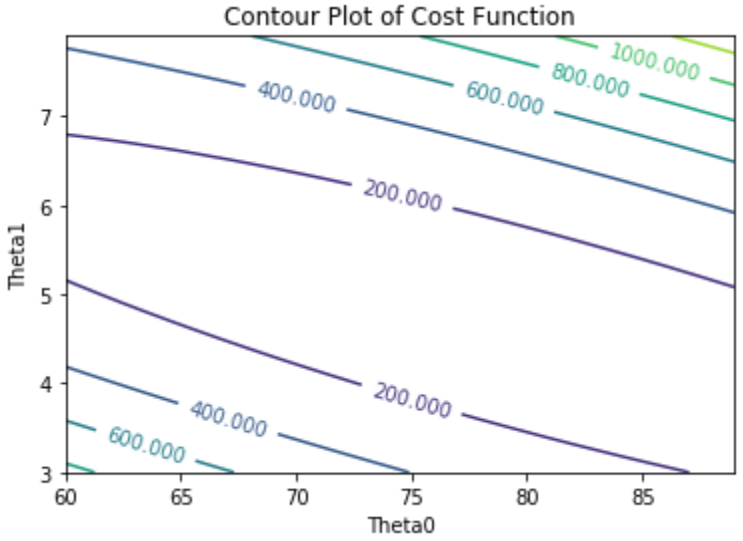
$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=0}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

where

- θ_0 and θ_1 are the parameters of our model
- m is the number of training examples
- y^i is the ith target value
- x^i is the ith feature vector, and
- h_θ is our model (hypothesis function of x) that we wish to evaluate.

Visualising the Cost Function – The Error Surface





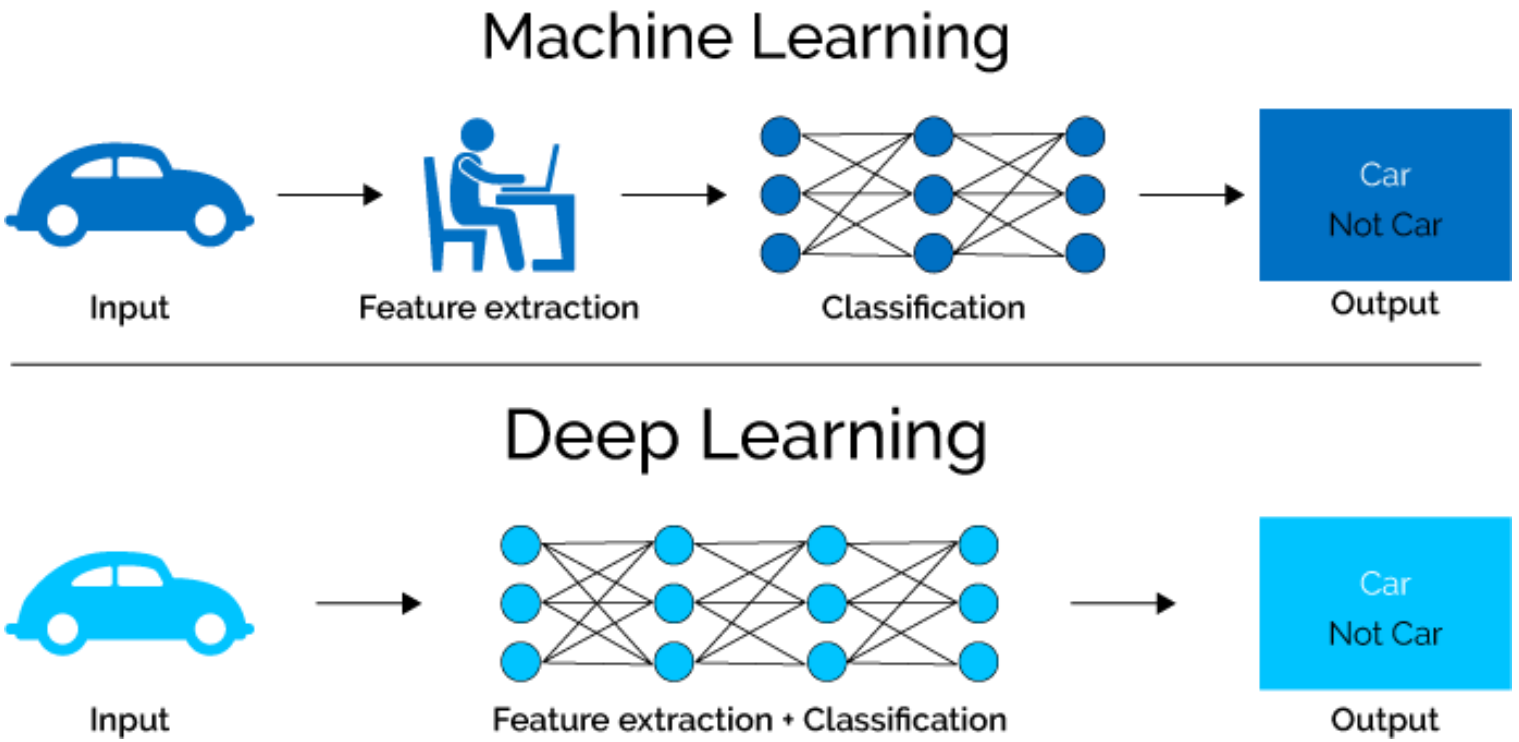
Towards Neural Solutions

The Problem with Hand Crafted Features



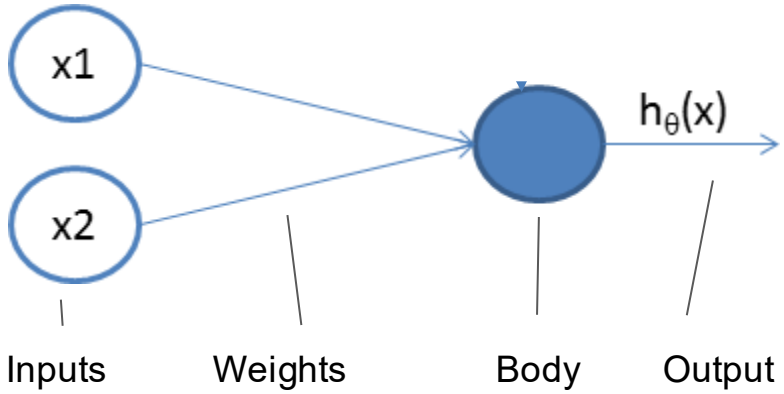
- Hard work
- Combinatorial explosion
- Often a fishing expedition

Avoiding Hand Crafted Features

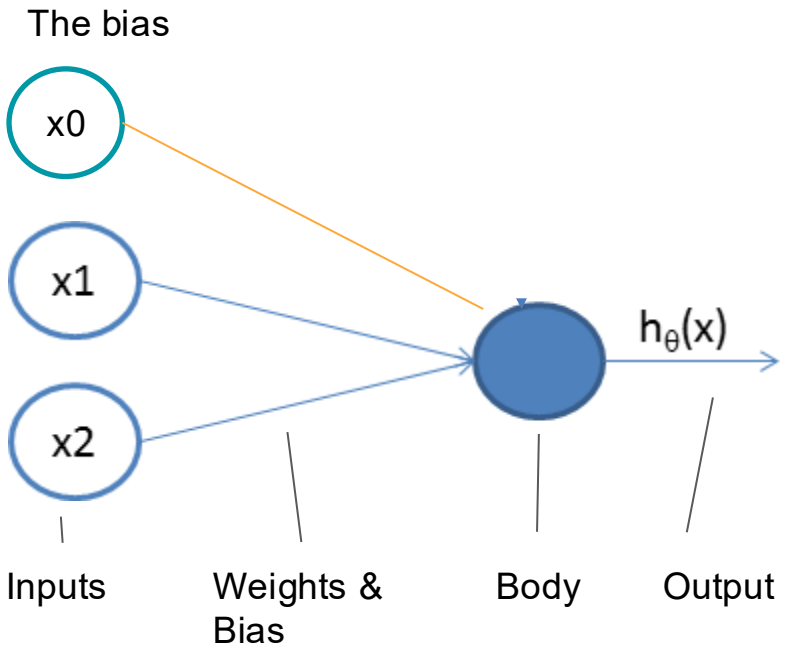


https://towardsdatascience.com/cnn-application-on-structured-data-automated-feature-extraction-8f2cd28d9a7e

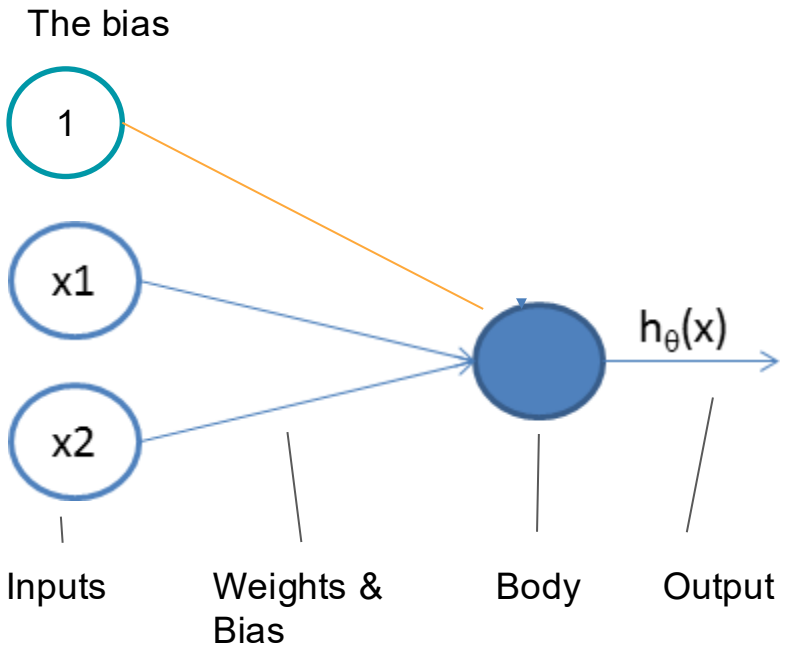
The Neuron



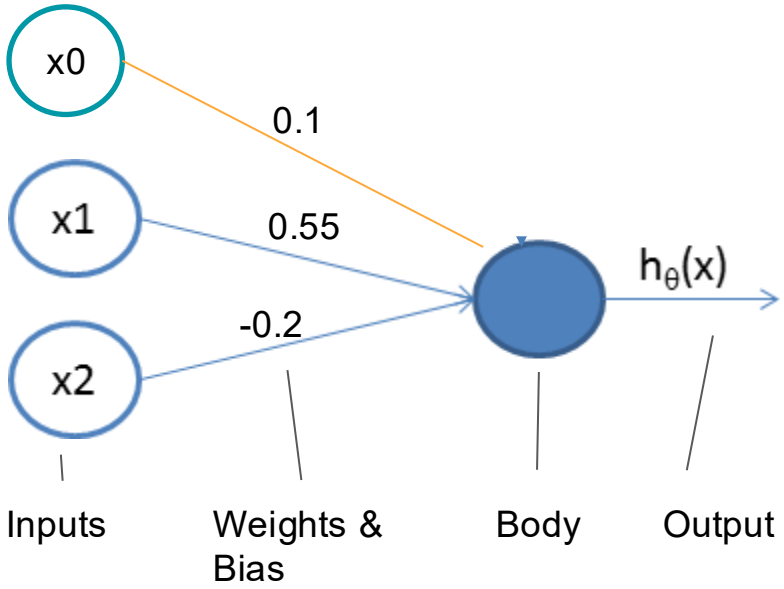
The Neuron



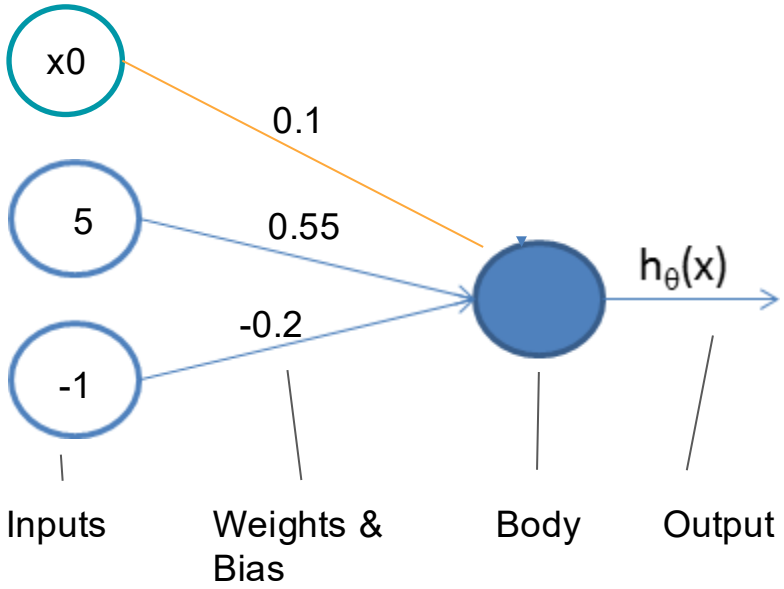
The Neuron



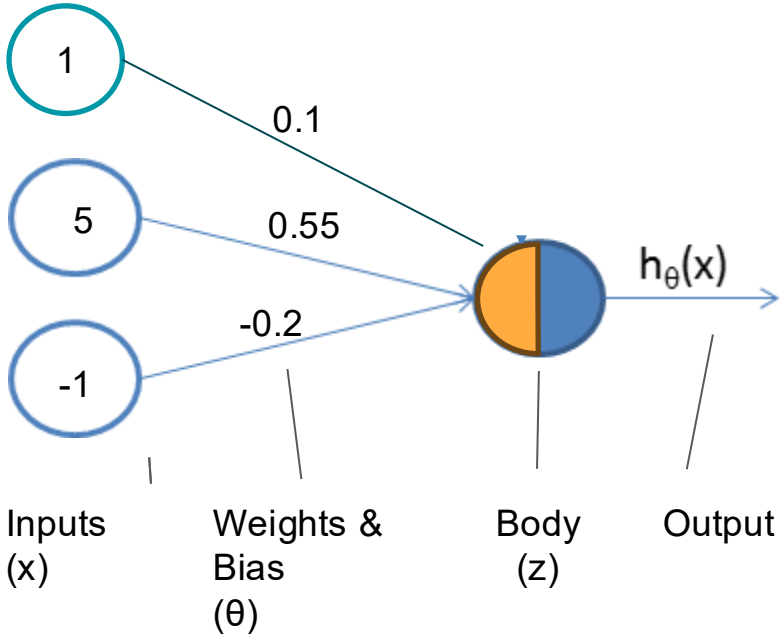
The Neuron



The Neuron



The Linear Neuron

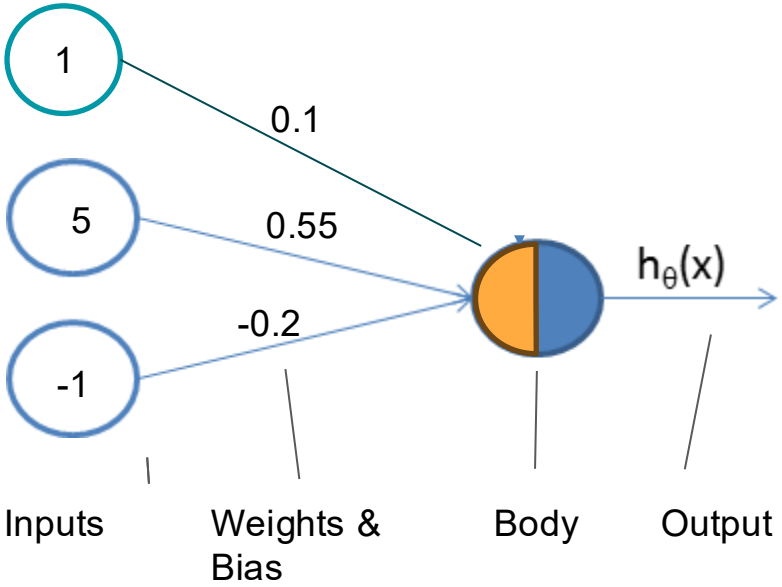


The Logit The Input

$$h_\theta(x) = z = \theta x$$

Output The parameters

The Linear Neuron

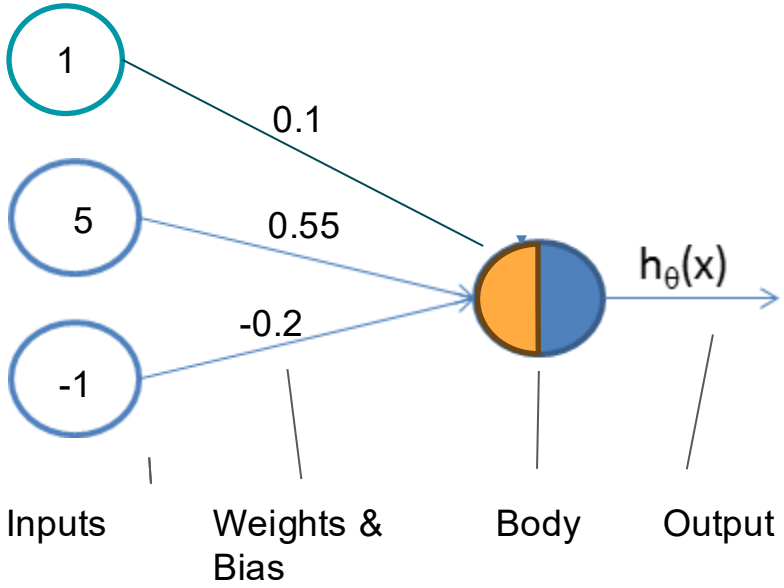


The Logit The Input

$$h_\theta(x) = z = \theta x$$

Output The parameters

The Linear Neuron



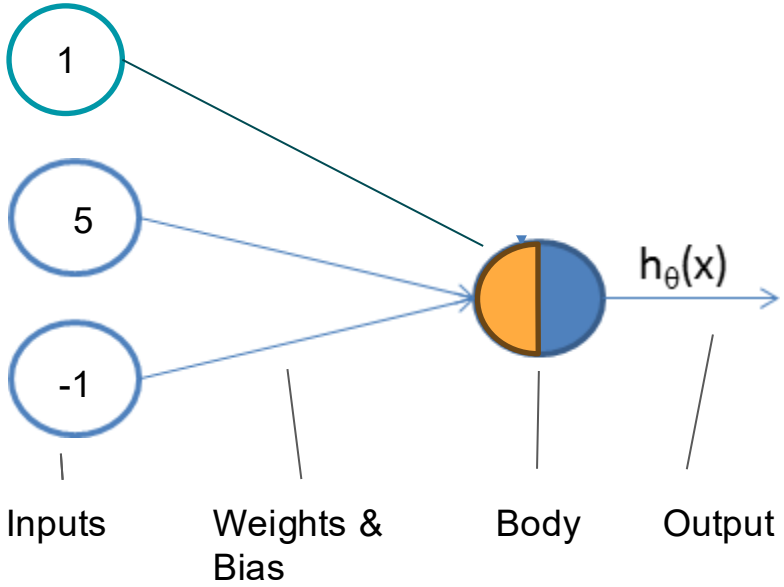
The Logit The Input

$$h_\theta(x) = z = \theta x = \sum_{i=0}^{n} \theta_i x_i$$

Output The parameters

The Logistic / Sigmoid Neuron

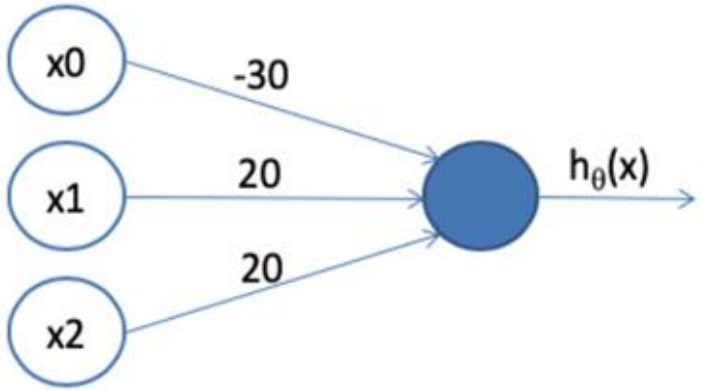
$$h_\theta(x) = \frac{1}{1 + e^{-z}}$$



$$z = \theta x = \sum_{i=0}^{n} \theta_i x_i$$

Some Hand Crafted Neurons

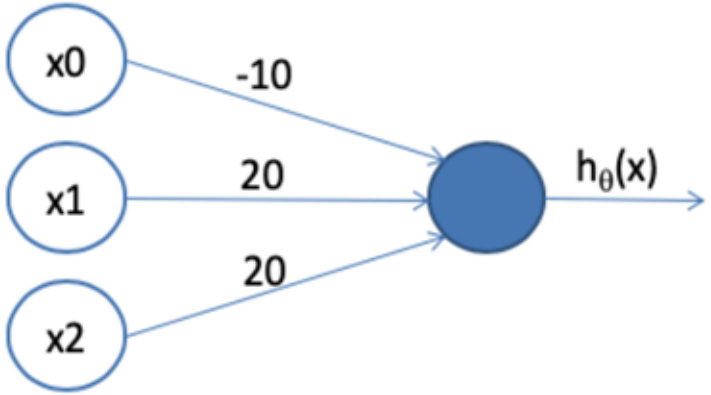
x1	x2	θx	h(θ x)
0	0	-30	0
0	1	-10	0
1	0	-10	0
1	1	10	1



AND

Some Hand Crafted Neurons

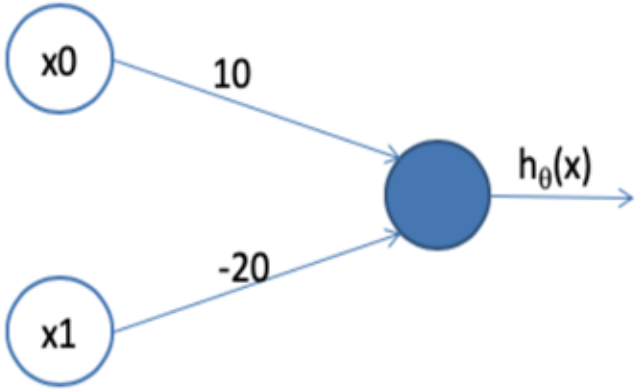
x1	x2	θx	h(θ x)
0	0	-10	0
0	1	10	1
1	0	10	1
1	1	30	1



OR

Some Hand Crafted Neurons

x1	θx	h(θ x)
0	10	1
1	-10	0

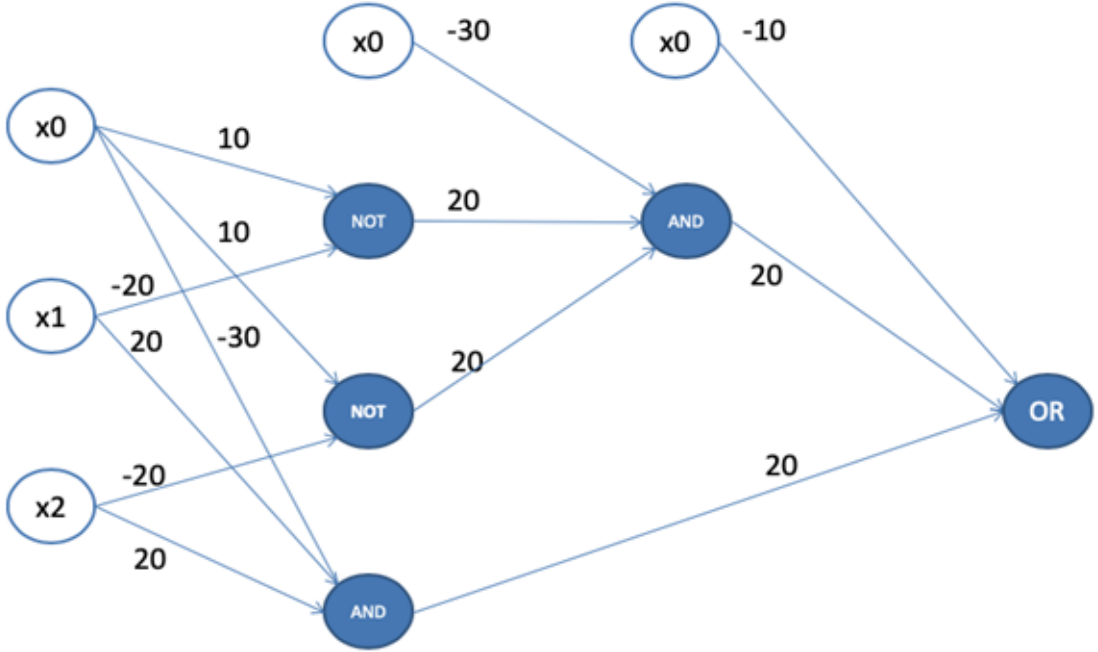


NOT

Building Something More Complicated

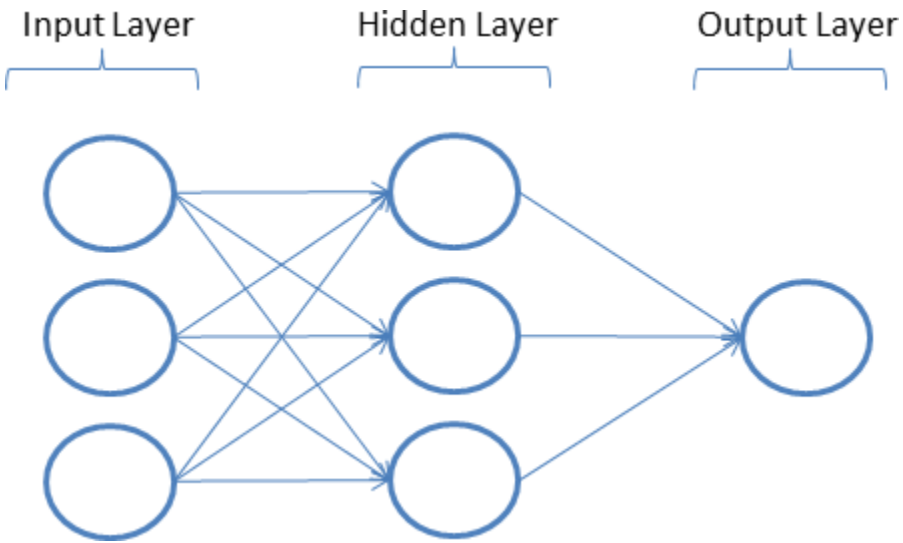
x1	x2	x1 XNOR x2
0	0	1
0	1	0
1	0	0
1	1	1

XNOR

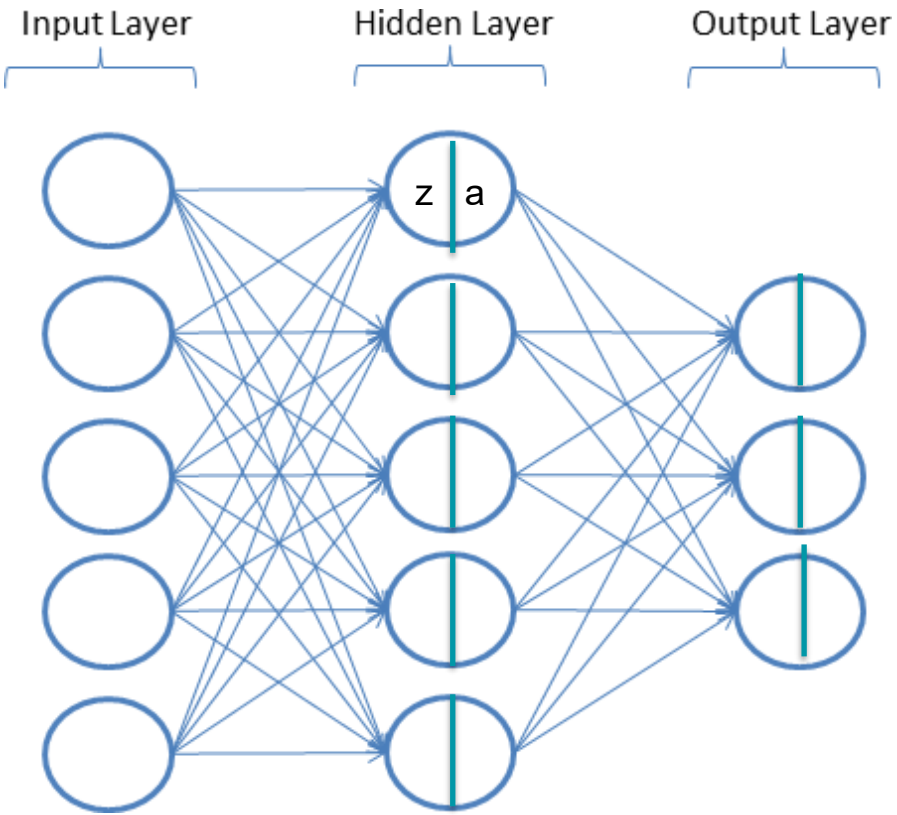


$$x1\ XNOR\ x2 = (x1\ AND\ x2)\ OR\ ((NOT\ x1)\ AND\ (NOT\ x2))$$

Layers of the Network - Feedforward Networks

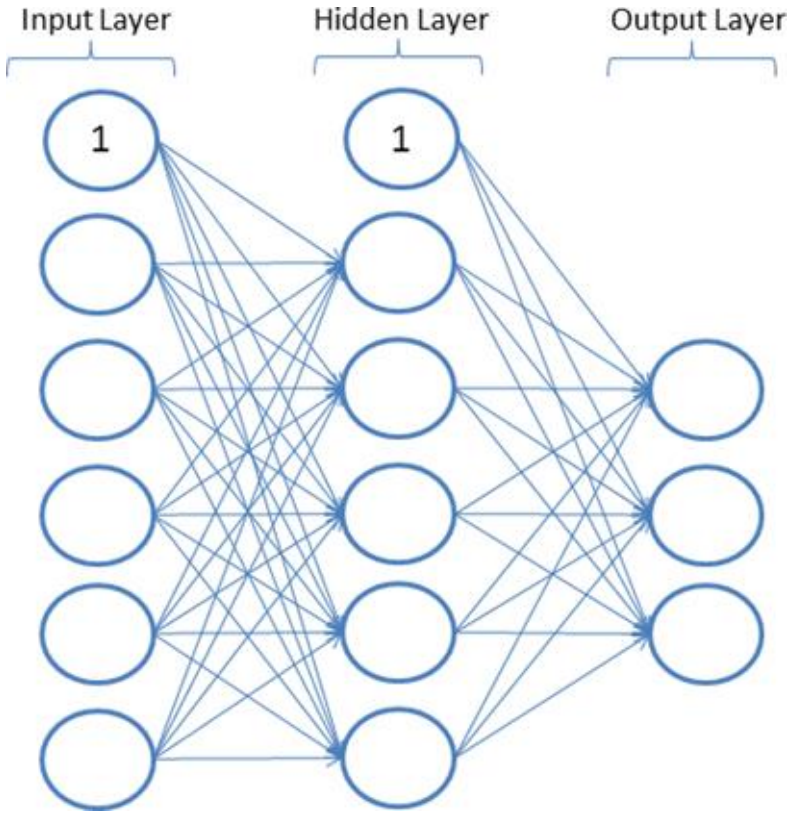


From Single to Multiple Targets

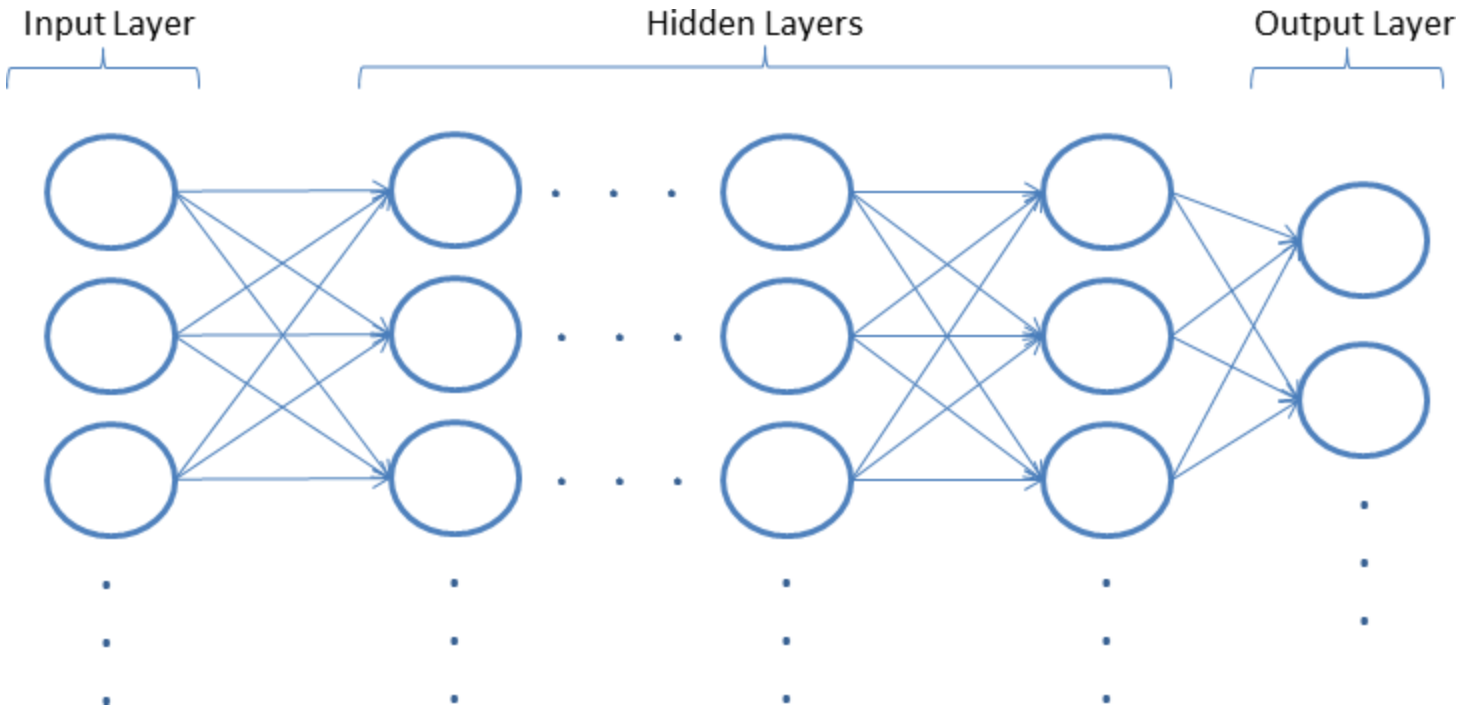


The Bias Unit

- Bias unit ‘data’ always 1
- The bias has a value - similar to a weight
- Output layer has no bias unit
- Bias is often not visualised



Deep Feedforward Networks



Aside: Note on Notation

- Data

$$Data_{Training} = \{(x^0, y^0), (x^1, y^1), (x^2, y^2) \dots (x^{m-1}, y^{m-1})\}$$

- Nodes

  a = activation or h

  z = logit

  a^l_u

  - a^0 refers to the vector of activation values for our input layer. This will correspond to our input vector x
  - a^{L-1} refers to the vector of activation values for our output layer. This will be our output vector y.
  - a^1 refers to the vector of activation values on our (first) hidden layer.
  - a^1_0 refers to the activation value of the bias unit on our (first) hidden layer. This will always be 1.
  - a^1_3 refers to the activation value of the 3rd (non-bias) unit in the (first) hidden layer as illustrated below.

- Weights

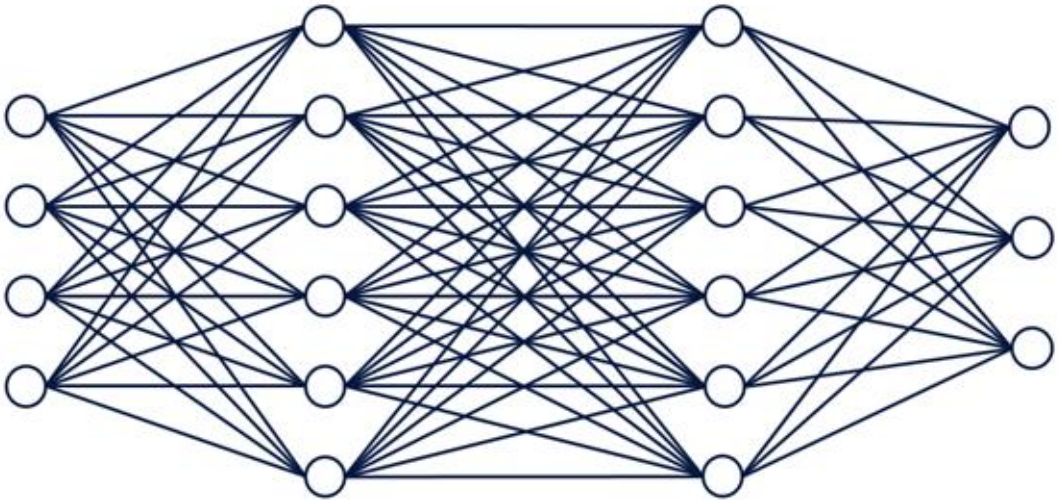
  - θ^0 refers to the weights emerging from the input layer
  - θ^{L-2} refers to the layer of weights feeding in to the output layer
  - θ^1 refers to the weights emerging from the (first) hidden layer
  - $\theta^1_{0,1}$ refers to the weight from the bias unit in the (first) hidden layer to the first unit in the following layer
  - $\theta^1_{5,2}$ refers to the weight highlighted in the figure below.

The Feedforward Algorithm

For Each Layer from the First Hidden Layer to Output

- Calculate z = (Input * Weights) + Bias
  - For first hidden layer input comes directly from input layer data
- Calculate Activation of neuron
  - Sigmoid Function or maybe just a linear function
- Pass output through as input to next layer
  - In the case of output layer we don’t have anywhere to pass the result but it can then be returned

The Feedforward Algorithm



https://community.alteryx.com/t5/Data-Science-Blog/Is-s-a-No-Brainer-An-Introduction-to-Neural-Networks/ba-p/300479

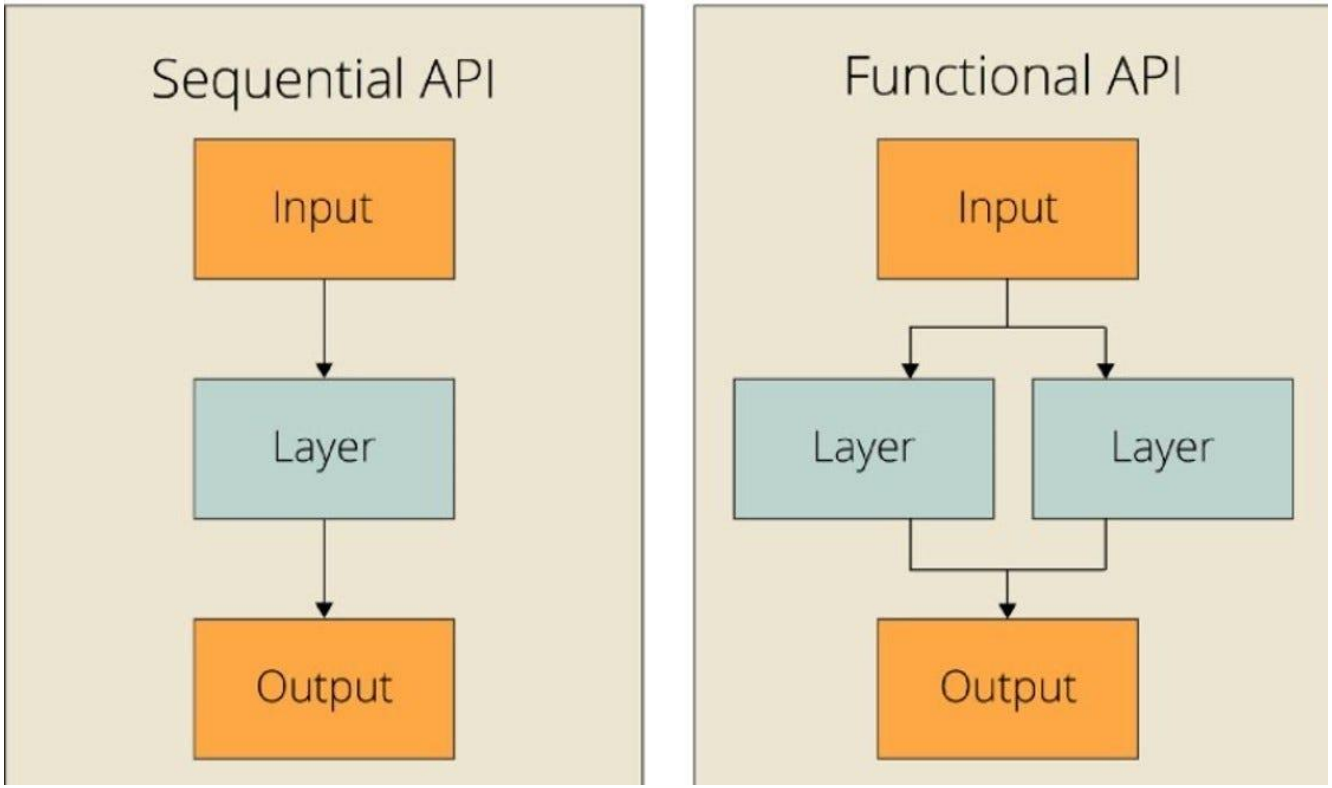
Network Design in Tensorflow - Functional

```
model = keras.Sequential()
model.add(layers.Dense(64, activation="sigmoid"), input_shape=(16,)))
model.add(layers.Dense(32, activation="sigmoid"))
model.add(layers.Dense(1))
```

Using the Functional Model

- Useful when you might have different routes through your network, or any other place where you might need to combine multiple elements

Functional Versus Sequential Models



Example of Functional Model

```
# Define the two distinct input layers
# The shapes must match the number of features in each path
input_A = keras.Input(shape=[5], name="wide_input")
input_B = keras.Input(shape=[6], name="deep_input")
hidden1 = layers.Dense(30, activation="relu")(input_B)
hidden2 = layers.Dense(30, activation="relu")(hidden1)
concat = layers.concatenate([input_A, hidden2])
output = layers.Dense(1, name="output")(concat)
model = keras.Model(inputs=[input_A, input_B], outputs=[output])
```

Sequential Vs Functional in Pytorch

```
import torch.nn as nn

# A simple sequential model
model = nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Linear(128, 10)
)
```

```
import torch
import torch.nn as nn

# A more complex functional-style model using
nn.Module
class MyModel(nn.Module):
    def __init__(self):
        super(MyModel, self).__init__()
        self.fc1 = nn.Linear(784, 128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        # The data flow is explicitly defined
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

Lab Work

See last week.

You should be comfortable using both Collab but also VS Code supported by github-copilot on a local install.

Review and execute the notebook for this week.